Introduction to Linux OS

By Mohamed Gamal

Standard Output and Error

In Linux and Unix-like operating systems the two primary channels that are part of the standard streams used for input and output operations are

1) **Standard Output (stdout)**
2) **Standard Error (stderr)**

Aspect	stdout	stderr
Purpose	Displays normal output	Displays error messages
File Descriptor	1	2
Default Output	Terminal	Terminal
Redirection	> 1> and >> ">" or "1>" to direct output by overwriting to the file ">>" to direct output by appending to the file	2>
	2>&1 or &> Redirects both **stdout** and **stderr**	

Example	Explanation
`$ echo "Hello World" > file.txt`	Direct output by overwriting to the file
`$ echo "World" >> file.txt`	Direct output by appending to the file
`$ ls -l /home/MG >> mg_list.txt`	Direct output by appending to the file
`$ ls MG 2> err.txt`	Direct error if exists to a file instead of screen
`$ ls nonexistentfile 2> /dev/null` `$ ls Documents/ Downloads/ Data/ 2> /dev/null`	
`$ ls MG/ Documents/ 2> err.txt 1> out.txt`	Direct error to err.txt and output to out.txt
`$ find /etc -type f -name passwd > /tmp/output 2> /dev/null`	Direct the standard output and the standard error
`$ find /etc -type f -name passwd >> /tmp/save-both 2>&1` `$ find /etc -type f -name passwd &> /tmp/save-both`	Save both output and error to the same file 2>&1 or &>
`$ ls Documents/ MG/ &> both`	

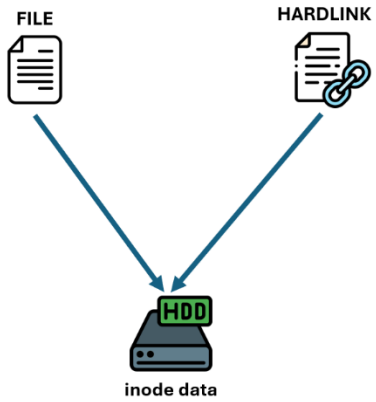
`/dev/null` is a special file that acts as a "black hole" (aka bit bucket); anything written to it is discarded, preventing error messages from being displayed on the terminal.

Hard and Soft (Symbolic) Links

In Linux, links are used to create references to files or directories. They can be of two types: **hard links** and **soft (symbolic) links**. Each type has specific characteristics, advantages, and use cases.

1. Hard Links

A hard link is essentially an additional name for a file. It points directly to the file's inode (the metadata of the file on disk), and multiple hard links to the same file share the same inode. It's used to provide multiple names for the same file in the same filesystem.



- **Direct Link to Data:** Hard links point to the same inode as the original file.
- **Independent:** If you delete the original file, the hard link remains and the data is still accessible.
- **Cannot Link Directories:** By default, hard links cannot be created for directories to prevent circular references and filesystem corruption (i.e., infinite loop); only for files.
- **Must Be on the Same Filesystem:** Hard links cannot span across different file systems since hard links reference the **inode** of a file, and inodes are **unique to a filesystem**.

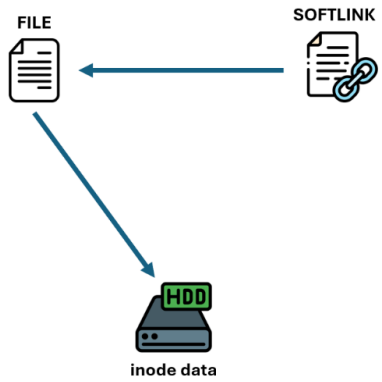
Hard links are useful in specific scenarios where you want multiple references to the same file while maintaining data consistency.

Command	Explanation
`$ ln original_file hard_link_name`	Create a hard link
`$ ls -li original_file hard_link_name`	Verify inode numbers

```
$ echo "Hello, World!" > original.txt
$ ln original.txt hardlink.txt
$ rm original.txt
$ cat hardlink.txt   # Data is still accessible
```

2. Soft (Symbolic) Links

A soft link is like a shortcut or alias. It contains a path to another file or directory.



- **Path-Based:** Symbolic links point to the file name (i.e., path), not the inode.
- **Breaks if the Target is Moved/Deleted:** If the original file is moved or deleted, the symbolic link becomes invalid (a "broken link").
- **Can Link Directories:** Symbolic links can point to directories.
- **Can Span Filesystems:** Symbolic links can reference files or directories on different filesystems or partitions.

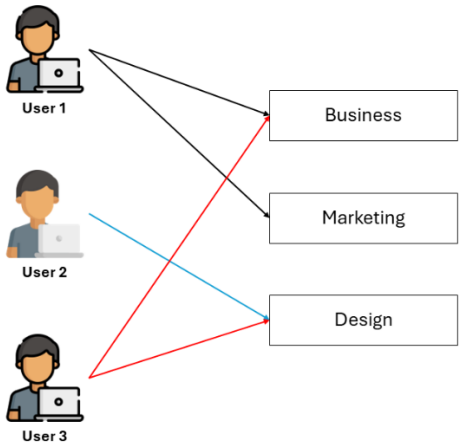
Command	Explanation
`$ ln -s target_file soft_link_name`	Create a symbolic link
`$ ls -li target_file soft_link_name`	Identify a symbolic link

```
$ ln -s /etc/hosts hosts_link
$ cat hosts_link    # accesses the /etc/hosts file
$ cp /etc/hosts /etc/hosts.bak
$ rm /etc/hosts
$ cat hosts_link    # broken link; error since /etc/hosts is deleted
```

Feature	Hard Link	Soft Link
Works for directories?	❌ No	✅ Yes
Works across filesystems?	❌ No	✅ Yes
Breaks if original is deleted?	❌ No	✅ Yes
Shares same inode?	✅ Yes	❌ No

Linux Users and Groups

Linux/Unix operating systems have the ability to multitask in a manner similar to other operating systems. However, Linux's major difference from other operating systems is its ability to have multiple users. Linux was designed to allow more than one user to have access to the system at the same time. In order for this multiuser design to work properly, there needs to be a method to protect users from each other. This is where permissions come into play.



Linux Primary and Secondary Groups

A **primary group** is the default group that a user account belongs to when created (/etc/group). Every user on Linux belongs to a primary group, usually stored in the `/etc/passwd` file.

```
$ cat /etc/passwd   or   $ tail -n 1 /etc/passwd
```

The `/etc/passwd` file in Linux and Unix-like systems stores basic information about each user or account. It is world-readable by default and can be viewed using tools like `cat` or a text editor.

Each line in the file represents a user and contains seven fields separated by colons:

- **Name:** The user's unique login name.
- **Password:** Originally stored an encrypted password, now replaced by `x` (actual passwords are stored in `/etc/shadow`).
- **User ID (UID):** A numeric identifier for the user; 0 is for root, and 100-999 are for regular users.
- **Group ID (GID):** The user's primary group ID, usually matching the UID.
- **GECOS:** General user information, like real name; often unused or empty.
- **Home Directory:** The user's starting directory upon login.
- **Shell:** The user's default shell (e.g., /bin/bash).

Once a user has been created with their primary group, they can be added to other **secondary groups** with a maximum of 15 secondary groups, usually stored in the **/etc/group** file.

--- Managing Users ---

Command	Description	Example
`$ adduser <username>`	Create a new user without setting a password.	`$ sudo adduser Ahmed`
`$ getent <entry> <username>`	View a user's GECOS info.	`$ sudo getent passwd Ahmed` `$ getent passwd`

`$ usermod <username>`	Modify a user account.	`$ sudo usermod -u 1100 Ahmed`
		`$ sudo usermod -l newAhmed Ahmed`
		`$ sudo usermod -d /home/New -m Ahmed`

`$ userdel -r <username>`	Delete a user and their home directory.	`$ sudo userdel -r Ahmed`
`$ passwd <username>`	Change a user's password.	`$ sudo passwd Ahmed`
`$ chfn <username>` `$ nano /etc/passwd`	Change user information (GECOS). ▪ **chfn:** change finger	`$ sudo chfn Ahmed` `$ sudo chfn -f "Ahmed" -o "" -p "" -h "" Ahmed`
`$ su <username>`	Switch to another user. ▪ **su:** switch user	`$ su Ahmed`

--- Managing Groups ---

Command	Description	Example
`$ groupadd <group name>`	Create a new group	`$ sudo groupadd Marketing`
`$ groupdel <group name>`	Delete (or remove) a group	`$ groupdel Marketing`
`$ usermod -a -G <group name> <username>`	Add a group to a user. ▪ **a:** append ▪ **G:** group	`$ usermod -a -G Marketing Ahmed`
`$ gpasswd -d <username> <group name>`	Remove a user from a group.	`$ gpasswd -d Ahmed Marketing`
`$ groups <username>`	View all groups a user in.	`$ groups Ahmed`

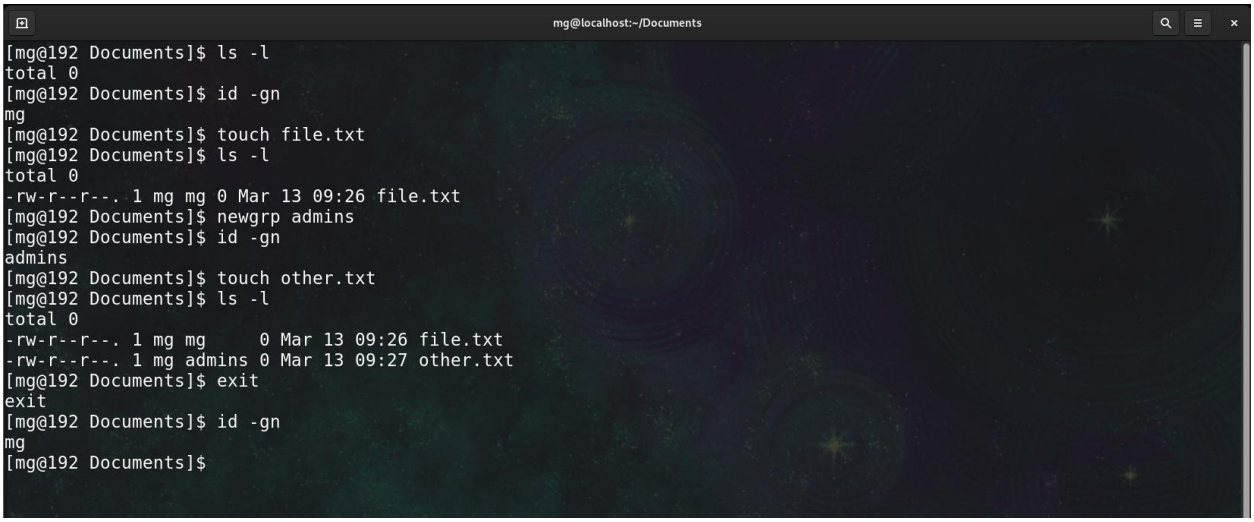
Temporarily Switching to a Group Session

You can temporarily switch to a group session to perform some tasks, when you use the `newgrp` command, you start a new shell session where the **Primary Group** of your user is temporarily changed to the specified group. You can also exit the new shell with `exit` command or **CTRL+D**.

```
$ newgrp <group name>
```

Example

```
$ newgrp Marketing
$ id -gn     # View current active group
$ exit  # Log out of the session
```



View all sudoers file and edit user permissions

The sudoers file is a configuration file that defines which users or groups have permission to run commands with elevated privileges (i.e., using sudo), and under what conditions, usually located at **/etc/sudoers**.

The **wheel** group is a **special administrative group** in **CentOS** (and other RHEL-based systems). Users in this group can **run commands as root** using sudo. Instead of giving root access to multiple users, you add them to wheel and control privileges via sudo.

Editing sudo Permissions for a User

Command	Description
`$ sudo -l` `$ sudo -l -U <username>`	List sudo Privileges for the current/specific user.
`$ sudo nano /etc/sudoers`	Directly edit sudoers file, however, editing /etc/sudoers manually can break sudo access if there are syntax errors!
`$ sudo visudo`	Safely edit sudoers file in the default editor Ahmed All=(ALL) NOPASSWD: /usr/bin/top, /usr/bin/ls » NOPASSWD: ability to run sudo **without a password**.

Edit Group Permissions

Command	Description

`$ sudo visudo`	Safely edit group permissions via sudoers file in the default editor.
	%Marketing ALL=NOPASSWD: /usr/bin/ls, /usr/bin/less, /usr/bin/apt
	» NOPASSWD: ability to run sudo **without a password**.
	`$ sudo usermod -a -G wheel ahmed`
	`$ sudo gpasswd ahmed wheel`